



SCS2204

University of Colombo, Sri Lanka

University of Colombo School of Computing

BACHELOR OF SCIENCE IN COMPUTER SCIENCE

Second Year Examination - Semester I - UCSC AY20 [held in October 2023]

SCS2204 — Functional Programming

(Two (2) Hours)

Answer ALL questions

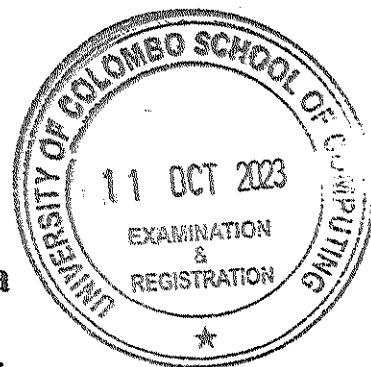
Number of Pages = 11

Number of Questions = 4

To be completed by the candidate

Index Number:

|  |  |  |  |  |  |  |  |
|--|--|--|--|--|--|--|--|
|  |  |  |  |  |  |  |  |
|--|--|--|--|--|--|--|--|



**Important Instructions to candidates:**

- I. Students should answer in the medium of English language only using the space provided in this question paper.
- II. Note that questions appear on both sides of the paper. If a page or a part of this question paper is not printed, please inform the supervisor immediately.
- III. Write your index number CLEARLY on each and every page of this Question paper.
- IV. This paper consists of 4 questions in 11 pages (including the Cover Page).
- V. Answer ALL questions.
- VI. Question 1 of the paper will total to 40 marks and the other questions will total to 20 marks each.
- VII. Calculators and any electronic device capable of storing and retrieving text including electronic dictionaries, smart watches and mobile phones are not allowed.
- VIII. Do not tear off any part of this answer book. Under no circumstances may this book, used or unused, be removed from the Examination Hall by a candidate.

**To be completed by the examiners**

|       |  |
|-------|--|
| 1     |  |
| 2     |  |
| 3     |  |
| 4     |  |
| Total |  |

### Question 1

- (a) Evaluate the following Scala programs and write down their final output. In the case of an invalid function write down as "Invalid". All questions are independent and therefore the result of one question will not affect the others.

|       | Function                                                                                                                                                                                                                         | Answer |
|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
| (i)   | <pre>object exam extends App{   def f1(x:Int,y:Int)=x*y   def f2(x:(Int,Int)=&gt;Int,y:Int)=x(y,y+1)+2   val x=f2(f1,5)   println(x) }</pre>                                                                                     |        |
| (ii)  | <pre>object exam extends App {   val x = List(1, 2, 3, 4, 5)   val f1 = (x: Int) =&gt; x * x   val f2 = (x: Int) =&gt; x + x   val y = x.map(x=&gt;f1(x)).map(x=&gt;f2(x))   println(y) }</pre>                                  |        |
| (iii) | <pre>object exam extends App {   val f1 = (x: Int, y: Int) =&gt; x * y   val f2 = (x: Int, y: Int) =&gt; x + y   val x = List(1, 2, 3, 4, 5)   val y = x.reduce(f1)+x.reduce(f2)   val z = x.map(x=&gt;x+y)   println(z) }</pre> |        |

|      |                                                                                                                                                                                                                                                                                                                                                    |  |
|------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|
| (iv) | <pre> object exam extends App {   val fx= (x:Int)=&gt;(y:Int)=&gt;     if(x%y==0) false else true   val lx= List(1,2,3,4,5,6)   val lfx=lx.filter(x =&gt; fx(x)(3))   println(lfx) } </pre>                                                                                                                                                        |  |
| (v)  | <pre> object exam extends App {   case class Rational(x: Int, y: Int) {     def numer = x     def denom = y     def op(r: Rational) =       new Rational(numer * r.denom + r.numer *         denom,denom * r.denom)     override def toString = numer + "/" +denom   }   val n1=Rational(1,3)   val n2=Rational(5,7)   println(n1.op(n2)) } </pre> |  |

(3\*5=15 Marks)

(b) Reduce the following lambda expressions.

|     | Lambda Expression.                  | Answer |
|-----|-------------------------------------|--------|
| (i) | $(\lambda x. \lambda y. + x y) 3 4$ |        |

|       |                                                                         |  |
|-------|-------------------------------------------------------------------------|--|
| (ii)  | $(\lambda x. (\lambda x. + (- x 1)) x 3) 9$                             |  |
| (iii) | $\lambda f. \lambda x. (f x) (\lambda y. + y 3) 4$                      |  |
| (iv)  | $\lambda f. \lambda x. \lambda y. (f y x) (\lambda a. \lambda b. b)$    |  |
| (v)   | $\lambda n. \lambda f. \lambda x. f (n f x) (\lambda f. \lambda x f x)$ |  |

(3\*5=15 Marks)

- (c) Consider the following functional program and identify all **Recursive Functions**, **Anonymous (Lambda) Functions** and **Higher-Order Functions** in the bellow table.

```
object exam extends App {
  def f1(a: Int, b: Int): Int = if (b == 0) a else f1(b, a % b)
  val f2=(x:Int)=>(y:Int)=> x*y
  val f3=(f:Int=>Int) => (x:Int)=>f(x)
  val L1=List(12,24,16)
  val f4=L1.reduce((x,y)=>f2(x)(y))
}
```

(10 Marks)

| Recursive Functions | Anonymous Functions | Higher-Order Functions |
|---------------------|---------------------|------------------------|
|                     |                     |                        |
|                     |                     |                        |

## Question 2

- (a) Any problems that you can solve using a looping will always have an alternative recursive solution. Write a recursive solution for the following Scala code:

```
for (a <- 1 to 10) println("Value of a: " + a)
```

(5 Marks)

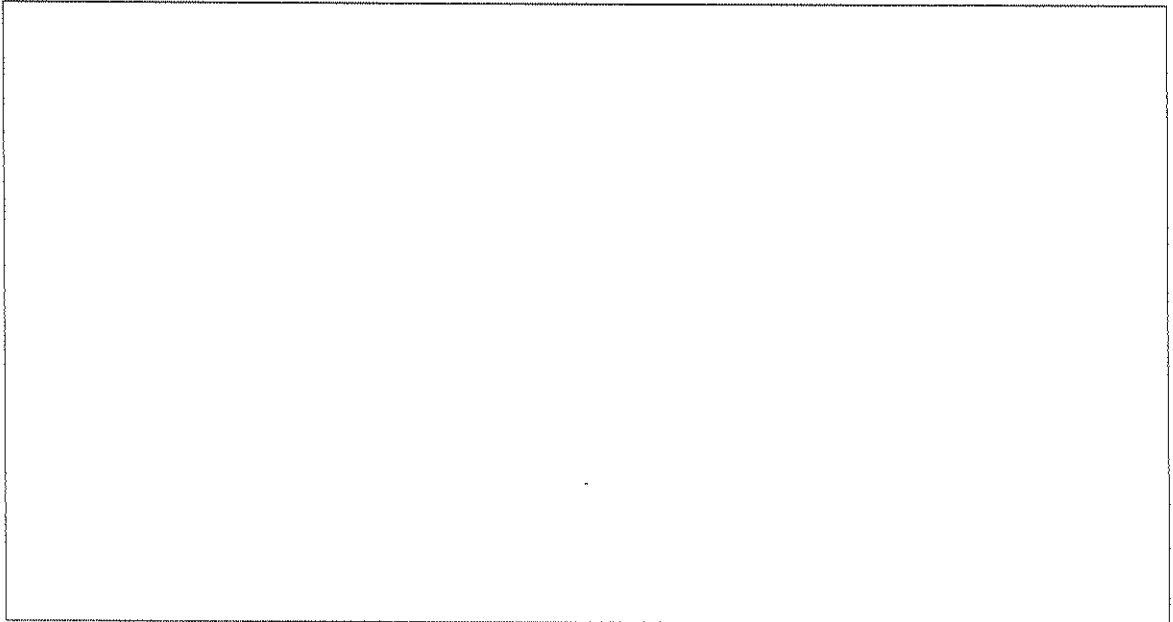
- (b) Two main types of the recursions are **direct recursion** and **indirect recursion**. Briefly describe these two recursion types with suitable examples.

(8 Marks)

- (c) The Towers of Hanoi is a mathematical puzzle whose solution illustrates recursion. There are three pegs which can hold stacks of disks of different diameters. A larger disk may never be stacked on top of a smaller. Starting with  $n$  disks on one peg, they must be moved to another peg one at a time. Write a recursive function to determine the smallest number of steps to move the stack for given  $n$ .

$$\text{hanoi}(n) = \begin{cases} 1 & \text{if } n = 1 \\ 2 \cdot \text{hanoi}(n - 1) + 1 & \text{if } n > 1 \end{cases}$$

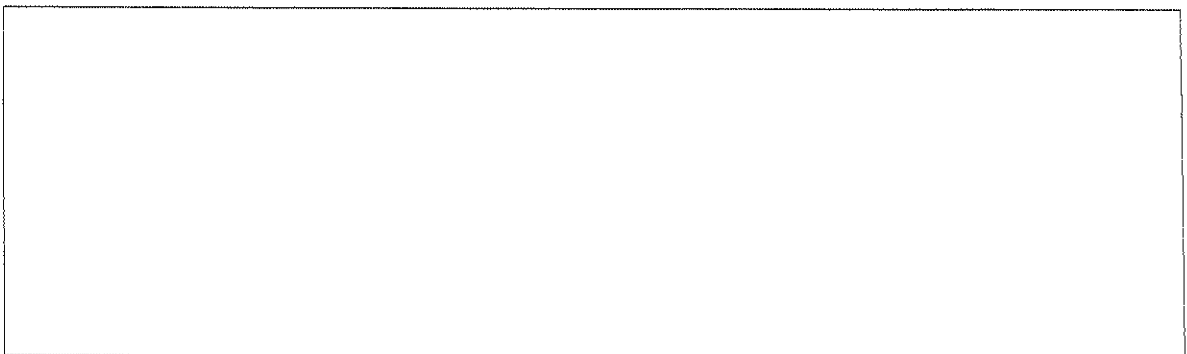
(7 Marks)



### Question 3

- (a) Define a lambda function that doubles the values in a given list. For example, input list (2,4,6,8) should be produced an output list (4,8,12,16).

(6 Marks)



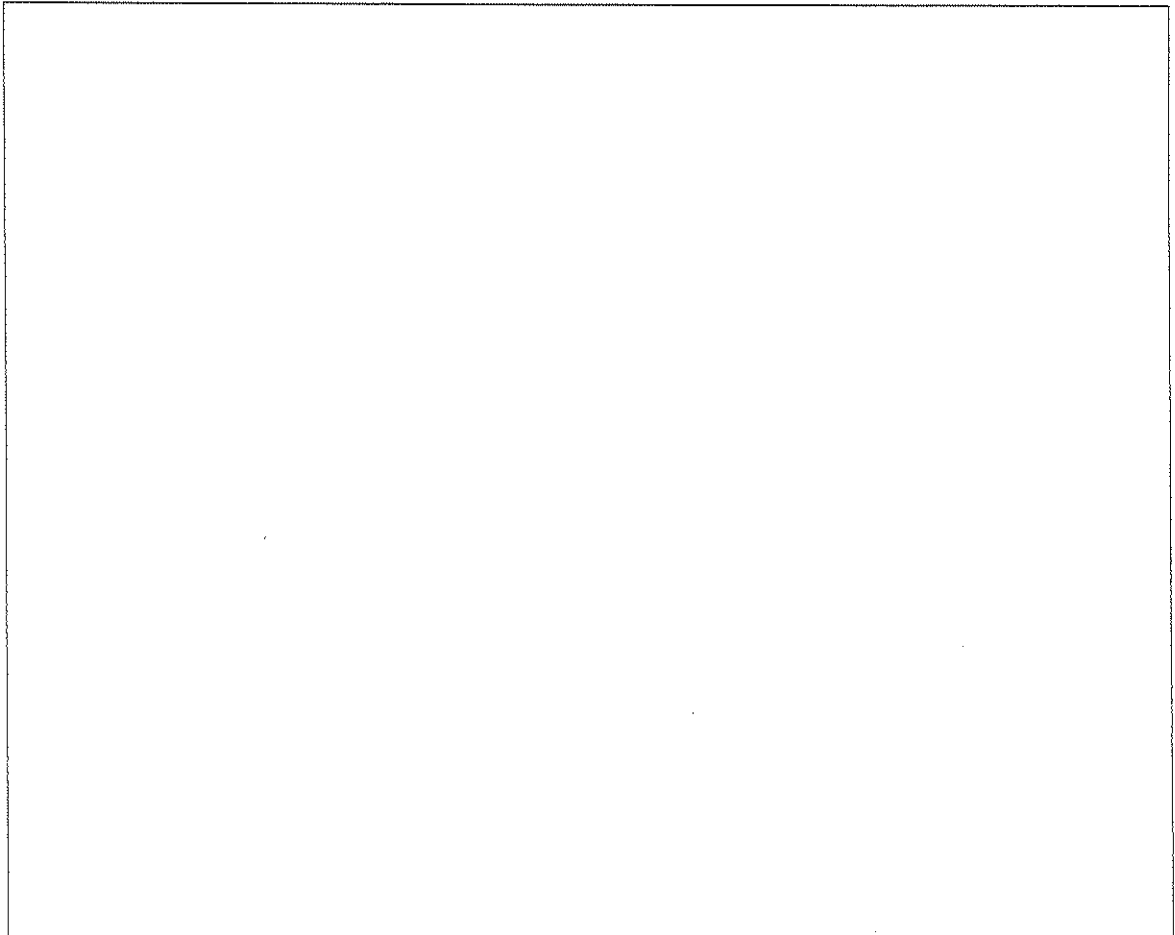




#### Question 4

- (a) Chamath wants to count a number of errors generated by his web application WebX. Chamath would like to use Apache Spark for this task. WebX logs all errors starting with a text “**ERROR**”. Suggest a suitable Scala Spark program for Chamath.

(8 Marks)



- (b)
- (i) Define an appropriate **case class** to keep the **name** of student and his/her exam **marks**.

(2 Marks)



- (ii) By using the above case class, create a list called students and store the following information.

| Name   | Marks |
|--------|-------|
| Nuwan  | 60    |
| Kulunu | 70    |
| Nimal  | 50    |
| Kasun  | 80    |

(3 Marks)

- (iii) Write a new function to calculate the total and average marks of the all students by using the map and reduce functions. By using this new function calculate and print the total and average marks as follows on the terminal.

Total marks =260

Average=65

(4 Marks)

- (iv) Provide the code to sort out the **students** list by marks in **descending** order and print them on the terminal as follows.

```
Kasun      => 80
Kulunu     => 70
Nuwan      => 60
Nimal      => 50
```

(3 Marks)



\*\*\*\*\*

